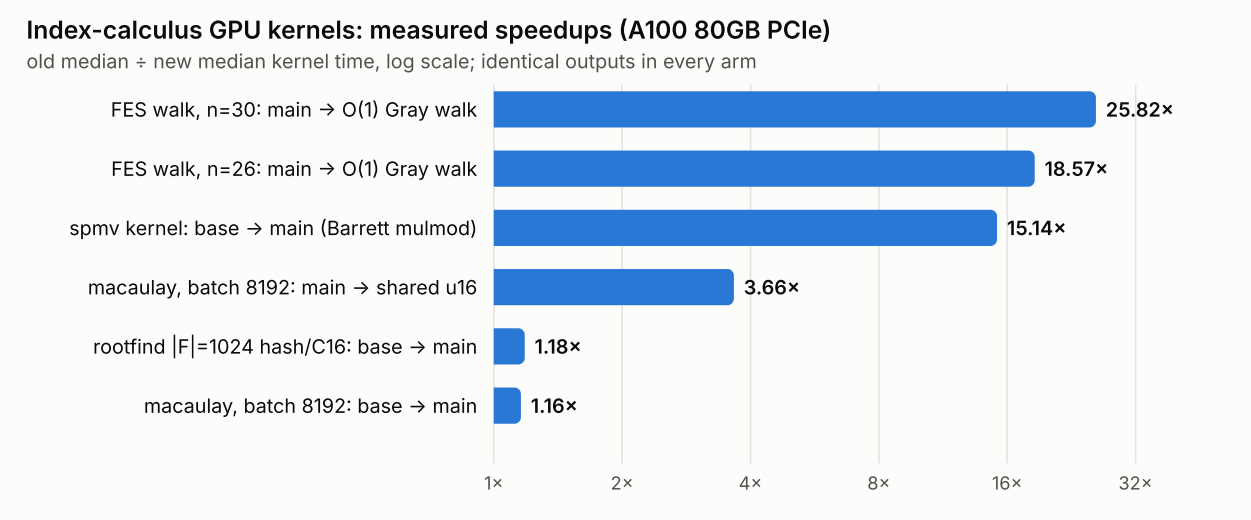
Index-calculus GPU kernels: hardware measurements and optimizations (rounds 1 and 2)

User request (2026-11-12): *"Work on optimizing my index calculus code for gpu."* Scope: the IC CUDA directories of docs/perf/20261112-ic-gpu-audit.md (`gpu/spmv`, `gpu/fes`, `gpu/macaulay`, `gpu/rootfind`, `gpu/semaev`). That audit merged four kernel commits with **no GPU run**. This study runs them, and three new changes, on one GPU.

Standing. Every number below is `[pcc:unverified]`: measured once, on one pod, by the session that wrote the code; no independent replay or validator has run. These are kernel-component speedups with **no rho arm**, so no row goes to `benchmarks/README.md` or the IC leaderboard (validation gate); they are not ECDLP-cost claims. The distance to matched rho is unchanged by this report, because no end-to-end IC solve was timed.



Kernel speedups on one A100, [pcc:unverified]

Host and method

- **GPU**: NVIDIA A100 80GB PCIe (sm_80, 108 SMs), driver 595.91.07, CUDA 12.8.93 (`nvidia/cuda:12.8.1-devel-ubuntu24.04`), Runpod pod `q1k8dm02q639jm` (Runpod-managed data centre), default boost clocks, 2026-11-12 23:12Z to 2026-11-13 00:29Z (~$2.70). Full `nvidia-smi -q`, `lscpu`, `nvcc` in `raw/out.tgz`. One hardware class; nothing here extends to sm_89/sm_90 without a run there.
- **Revisions**: base `e9d5dbdccd` (before the audit), main `7fd7c580b6` (with the audit's four commits), final `c80afaa98e` (this branch). Trees shipped as `git archive` tarballs, SHA-256 checked on upload (`rp_gpu.sh`, a GPU variant of `research/runpod_kit_20261112`).
- **Timing**: kernel-only. macaulay and rootfind use their benches' CUDA-event timers. spmv and fes print no time, so every arm's working copy gets the same two inserted lines (`steady_clock` before the launch, print after the following `cudaDeviceSynchronize`; `job.sh: shim`), with `CUDA_MODULE_LOADING=EAGER`. Five rounds per arm, arms interleaved in a fixed rotation, medians

reported. Each comparison has an **A/A arm**: the same source built a second time and timed in the same rotation.
- **Identity**: on the device, every arm runs its directory's selftest against the host reference, and the workers' sorted outputs are hashed against the CPU workers on the same inputs.
- **Bookkeeping**: record `benchmarks/records/20261112T2313Z-ic-gpu-cards.json`. The S3 resume checkpoint and S3 raw upload were not possible (the session's AWS keys are rejected); the user waived the checkpoint for this run, and the raw data (32 KB) is in `raw/` with `raw/SHA256SUMS`.

1. The audit's four commits on hardware (base → main)

kernel	metric	base	main	ratio	A/A
spmv	kernel ms, 2^16 rows × 32 nnz × 32 lanes	8.175	0.540	**15.1× faster**	0.999
fes	kernel ms, n=26, default worker mode	69 200	7.28	~9 500× (bug fix)	1.000
fes	kernel ms, n=26, same split (`--low-bits 10`)	6.45	7.28	**1.13× slower**	0.998
macaulay	ms per batch of 8192 (226×286, p=2083)	831.7	718.2	1.16× faster	1.000
macaulay	ms per batch of 64	12.94	9.28	1.39× faster	0.995
rootfind	ms, |F|=1024, hash, CHUNK 16	23.53	19.91	1.18× faster	1.002
rootfind	ms, |F|=65536, every config	—	—	0.995–1.000	≤1.007

Correctness on the device:

- spmv: CPU, base and main output digests equal (`d9aafdd3…`).
- fes, n=16 m=3 (8256 solutions on the CPU): **base returns 64**, the per-thread cap the audit predicted; main returns all 8256, digest equal to the CPU. Compiled-in selftests pass on both.
- macaulay `selftest 64`: 0 of 64 instances differ, both revisions.
- rootfind `selftest` (|F|=1024 and 4096) and `p256`: pass on base and main for all six (lookup, chunk) configurations.

The audit's shared-memory FES walk table is the one regression: same split, 13% slower. It is superseded by section 3.

2. Build breaks found on hardware (fixed)

- **rootfind `bench` never linked under nvcc** (base and main): `fp64_inv` / `fp256_inv` are `__noinline__` functions defined in a header without `inline`, so `bench.cu` and `rootfind_cuda.cu` both define them (`ld: multiple definition`). Fix: `RF_BIG` gains `inline` (`fp64.cuh`). Linkage only; both rootfind arms in section 4 were built with it.
- **semaev default build fails on CUDA 12.8** (audit finding 10, confirmed): ptxas rejects `clmad.lo/hi`. Fix: `GF2N_CLMAD` defaults on only for sm_80+ *and* CUDA ≥ 13.3 (the version the repository's 22.4% clmad measurement used); 12.x and 13.0–13.2 take the portable path (same results). Default build and device selftest pass on CUDA 12.8 (`raw/out_fes.tgz`).
- **spmv worker had no `-arch`**: nvcc 12.x then embeds only sm_52 code and the driver JIT-compiles it. `ARCH ?= all-major` (native SASS per major architecture plus PTX); `cuobjdump` lists 14 ELF images in the final build.

3. FES: O(1) Gray-code walk (new)

`fes_walk_sym` refreshes all n first derivatives at every step. `fes_walk_gray` keeps them lazily (the bookkeeping of Bouillaguet et al., CHES 2010): between two flips of variable k, every lower variable flips an even number of times and one higher variable k′ flips once, so D_k changes by exactly `qsym[k][k′]` with `k′ = ctz(s & (s-1))`; before the first flip of k, by `qsym[k][k-1]`. Per point: one shared-memory load and two XORs instead of n loads and n XORs. Every value the walk computes equals the old walk's at the same step, so the zeros are the same points **in the same order**.

Host check (`make test`): emitted lists equal to `fes_search_subcube` on every prefix at splits 0..12 and on tiny sub-cubes (low_bits 0..3), on 7 systems (n = 1, 2, 6, 12, 16, 18, 20; up to 212 160 zeros). Device: selftest MATCH; sorted outputs equal to the CPU on n=16 (8256 solutions), n=24 m=12 (4163), n=26 and n=30.

input	main (ms)	Gray walk (ms)	speedup	A/A (sym walk, new tree)
n=24, m=12	1.683	0.149	11.3×	0.999
n=26	7.271	0.392	18.6×	1.001
n=26, `--low-bits 10`	7.279	0.394	18.5×	1.000
n=30	132.8	5.14	**25.8×**	1.006

Against base at the same split (section 1, same pod, different job): 6.45 → 0.394 ms, 16.4×.

4. Rootfind: fused unwind + sweep (new, rejected)

The audit found `d[]` in local memory because the sweep indexes it at run time. Variant R1 fuses the sweep into the back-substitution loop (compile-time indices), R2 does the same with the per-entry body as a `__noinline__` call. Both pass every device selftest. Timing (medians, ms):

|F|	best old config	R1 best	R2 best
65536	sorted/C16 401.9	sorted/C32 491.5	sorted/C64 594.5
1024	hash/C16 19.91	hash/C16 23.96	hash/C16 21.56

R1 is faster than the old kernel at some configurations [pcc:unverified] (hash/C64 at |F|=65536: 0.75×), but never the best one, and it is 2.8× slower at |F|=1024 hash/C64. Not merged; the patch is `rootfind-fused-sweep-rejected.patch`, full tables in `RESULTS.md`.

5. Macaulay: shared-resident u16 matrix (new)

The global kernel reads and writes the whole 258 KB matrix per pivot column. With p < 2^16 every Montgomery residue fits in 16 bits, so the matrix plus scratch (132 480 B) fits in sm_80's 163 KB opt-in shared memory. `rref_batch_kernel_smem` loads it once, runs the **same** `mac_rref_block` (templated on the storage type; all arithmetic stays 32-bit, only stores narrow) and writes it back once. `bench.cu` falls back to the global kernel when the device cannot hold it (sm_86/89: 99 KB) or p ≥ 2^16; `MAC_SMEM=0` forces global.

Device selftest (digest, rank, pivots vs the host serial reduction): pass at 256/512/1024 threads, on 226×286 p=2083 and 64×80 p=271.

batch	main global (ms)	shared 256	shared 512	shared 1024	1024 vs main	A/A
64	9.30	5.47	3.30	2.55	3.65×	1.000
1024	92.0	54.7	33.2	25.8	3.57×	0.999
8192	718.3	417.4	253.4	196.5	**3.66×**	1.000

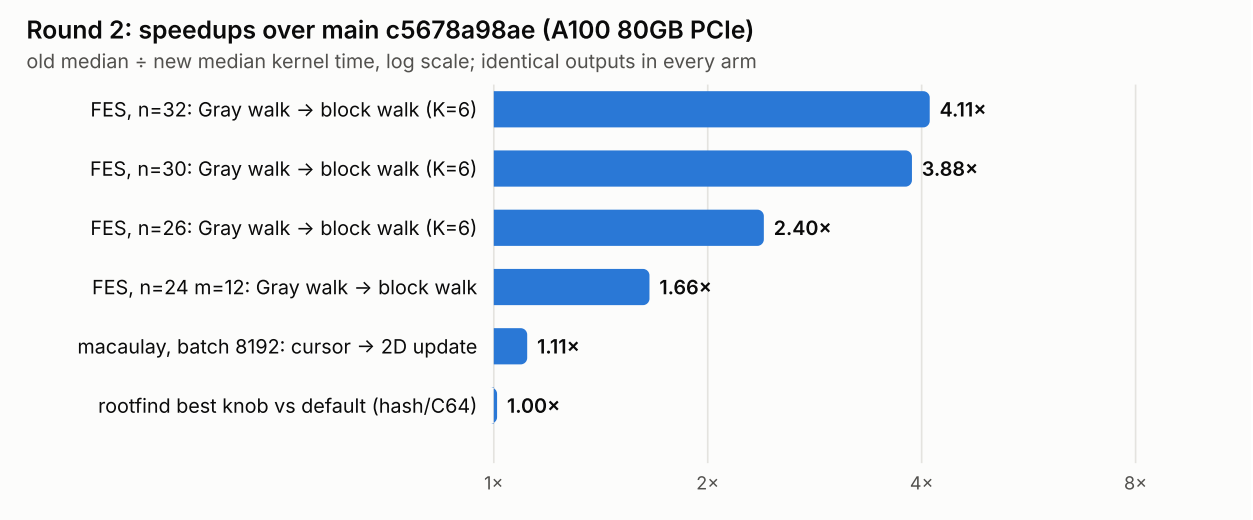
Default set to 1024 threads. Final committed tree: 195.9 / 195.8 / 196.0 ms at batch 8192. Against base: 831.7 → 196 ms, 4.2×.

Final check of the committed tree (`c80afaa98e`)

Built with each Makefile's defaults (plus `ARCH=sm_80`): all host tests pass; fes, macaulay and semaev device selftests pass; worker digests equal the CPU for spmv and fes n=16/24/26/30; semaev default build succeeds. `raw/out_final.tgz`.

Round 2 (2026-11-13): main `c5678a98ae` → `ffebecd472`

User: *"More improvements."* Same machine class and the same physical host as round 1 (Runpod machine `bwxqa7t7y2yk`, A100 80GB PCIe, CUDA 12.8), pod `o6oa4ymjxo2sgz`, 00:57Z–01:17Z (~$0.75). Record `benchmarks/records/20261113T0057Z-ic-gpu-round2.json`; raw `raw/out2.tgz`, `raw/out_fin2.tgz`; tables `RESULTS2.md`. Rootfind builds ran on the pod's CPUs while FES and macaulay were timed; the timers are GPU-side and every A/A arm ran under the same load. Round 1's numbers reproduce on round 2's main arm: FES n=26 0.392 → 0.392 ms, macaulay batch 8192 196.5 → 197.1 ms.



Round 2 speedups, [pcc:unverified]

R2.1 FES: block-unrolled walk (new, default K = 6)

`fes_walk_block<K>` cuts the Gray walk into blocks of 2^K steps. Inside a block the flipped variable and its partner are compile-time constants (template recursion over the 2^K − 1 steps), so D_0..D_{K−1} and their K(K−1)/2 cross terms live in registers: no `ctz`, no table or local-memory load per step. One step per block (variable K + ctz(h)) and K column loads stay dynamic. The values are `fes_walk_gray`'s step for step, so the zeros come out in the same order.

Identity: host `make test` on 6 systems (n = 1...20, up to 244 800 zeros) at splits 0..14; device sorted outputs equal to the CPU on n = 16, 24 (m=12), 26, 28 (m=16), 30, and equal to main on n = 32 (no CPU reference at 2^32). K = 8 does not compile (nvcc template depth); capped at 7.

input	main Gray (ms)	K=4	K=5	K=6	K=7	A/A
n=24, m=12	0.151	0.098	0.092	**0.091** (1.66×)	0.098	0.997
n=26	0.392	0.180	0.168	**0.164** (2.40×)	0.180	1.003
n=28, m=16	1.328	0.462	0.415	**0.398** (3.34×)	0.449	0.999
n=30	5.043	1.552	1.377	**1.301** (3.88×)	1.469	1.000
n=32	19.85	5.853	5.142	**4.834** (4.11×)	5.828	1.000

Cumulative, n=30: 132.8 ms (round 1 main) → 5.14 ms (Gray) → 1.30 ms (block), **102×**, same solution list.

R2.2 Macaulay: 2D update mapping (new, default on the shared path)

Lanes take columns (the pivot-row value stays in a register for the whole column), warps take rows (`factor[i]` is a warp-wide broadcast); no flattened cursor. Selftests pass on both shapes.

batch	main (ms)	2D, 1024 thr	2D, 512 thr	A/A
64	2.55	2.30 (0.902)	2.84	1.000
1024	25.89	23.18 (0.895)	28.61	1.000
8192	197.1	**176.9 (1.11× faster)**	218.1	1.000

Default: 2D on the shared u16 path; the global u32 fallback (256 threads, sm_86/89) keeps the cursor because 2D was not measured there. Final tree: 176.7 ms ×3. Against base `e9d5dbdccd`: 831.7 → 176.7 ms, 4.7×.

R2.3 Rootfind launch knobs (measured, **no change**)

`RF_BLOCK` ∈ {64, 128, 256} × `RF_MIN_BLOCKS` ∈ {0, 2}, on main's source; all pass the selftests. The default (128, 0) is best or within 1.5% everywhere; the only gain outside A/A is −1.5% for (128, 2) on |F|=1024 hash/C16, which costs 9–46% on other configurations. Defaults kept. Nsight Compute is unavailable on the pod (`ERR_NVGPUCTRPERM`), so no profile explains where rootfind's time goes.

Not done / next

- No IC pipeline code calls the FES worker except the `icx fes` demo (`src/cryptanalysis/fes_gpu.rs`), so an end-to-end relation-phase number does not exist yet; at 0.16 ms of kernel per n=26 system, the worker's process start and text I/O dominate that path.
- Other GPU classes (sm_89 cannot take the macaulay shared path; sm_90 can, with more room).
- End-to-end effect: none of these kernels was timed inside an IC solve, so no ECDLP-cost or rho-gap statement is made. The relation phase that calls FES (`fes_gpu.rs`) is the obvious place.
- Independent replay (validator) before any of this is cited as more than `[pcc:unverified]`.
- rootfind: |F|=65536 is dominated by sorted lookup; the header's "global hash" idea is untested.
- The Metal FES worker still runs the O(n) walk.